

CUSTOMER CHURN & RETENTION ANALYTICS SYSTEM

Project Report

Customer Churn & Retention Analytics System



Technology Stack: Python, MySQL, Machine Learning, Power BI, Streamlit
Domain: Customer Analytics & Business Intelligence

Author: Bimlesh Kumar Sah
Version: 1.0

Email: sahbimlesh9@gmail.com

GitHub: <https://github.com/sahbimlesh00>

1. Executive Summary

Customer churn is one of the most significant challenges faced by subscription-based and service-oriented organizations. Acquiring new customers is substantially more expensive than retaining existing customers. Therefore, identifying customers who are likely to leave and implementing proactive retention strategies can significantly improve profitability and business growth.

The Customer Churn & Retention Analytics System is an end-to-end analytics and machine learning solution designed to identify churn patterns, predict customer attrition, quantify revenue at risk, and provide actionable business insights.

The solution integrates MySQL for data storage, Python for data preprocessing and machine learning, Power BI for business intelligence reporting, and Streamlit for predictive analytics deployment. The system enables business stakeholders to make data-driven decisions and prioritize customer retention efforts.

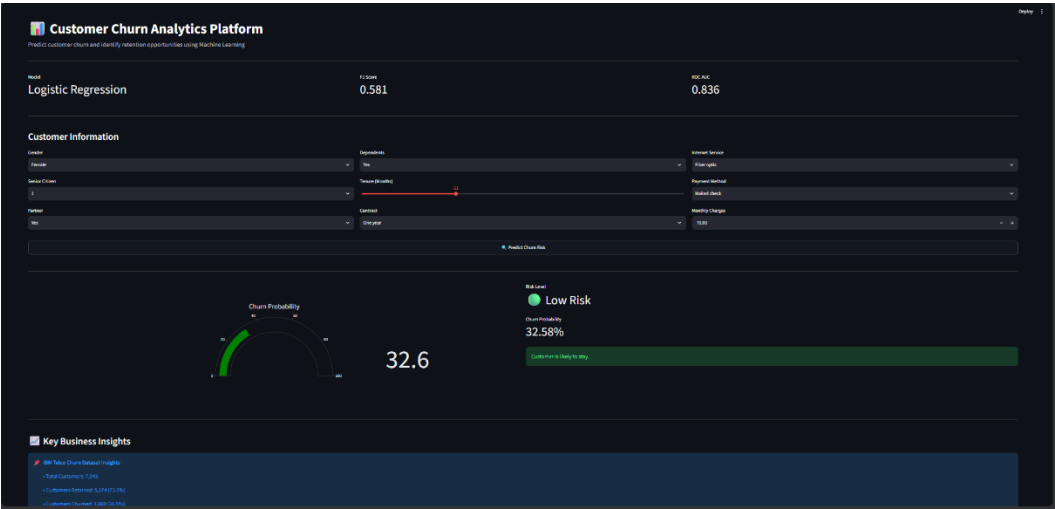
Metric	Value
Customers analysed (after cleaning)	7,032
Churned customers	1,869 (26.58%)
Monthly recurring revenue (all customers)	455,661
Monthly revenue lost to churn	139,130.85 (30.5% of MRR)
Revenue lost from month-to-month contracts	120,847.10 (86.9% of all lost revenue)
Selected model	Logistic Regression, ROC-AUC 0.836, F1 0.581, accuracy 79.6%

2. Business Problem Statement

Customer churn directly impacts recurring revenue, customer lifetime value, and overall business sustainability.

Organizations often face challenges such as:

- Inability to identify customers likely to churn
- Lack of visibility into churn drivers
- Revenue loss due to customer attrition
- Absence of predictive decision-making tools
- Inefficient allocation of retention resources



1. Data Flow

The Customer Churn & Retention Analytics System processes raw telecom customer files into a governed SQL data layer, an exploratory/predictive Python pipeline, an executive Power BI dashboard, and a Streamlit predictive web application. Analyzing a cleaned dataset of **7,032 customers**, the system identifies an overall churn rate of 26.58%, representing 1,869 lost customers. Across the customer base, the total monthly recurring revenue is 455,661, with 139,130.85 lost to churn. The data reveals that 86.9% of this lost revenue originates specifically from month-to-month contracts.

The end-to-end data flow operates sequentially across five stages:

- **Ingestion:** The raw Telco-Customer-Churn.csv file, containing 7,043 rows and 21 columns, is manually loaded into a MySQL staging table named stg_customer_churn.
- **Cleaning and Semantic Layer:** SQL scripts extract, trim, cast types, and remove 11 blank-tenure rows to create a clean customer_churn table. A semantic layer of 11 views, including the primary vw_dashboard_fact, is generated to serve downstream consumers.
- **Analytics and Training:** A Jupyter notebook connects to the database via SQLAlchemy and PyMySQL, performs Exploratory Data Analysis (EDA), segments customers, and applies a stratified 80/20 train/test split. A Logistic Regression model is trained and outputs four artifact files (.pkl).
- **Consumption:** Power BI connects directly to vw_dashboard_fact for business visualizations, while the Streamlit app loads the pickled artifacts to score individual customer risk profiles interactively.
- **Business Outcome:** The insights and prediction tools allow stakeholders to size retention strategies, visualizing how a 10% retention improvement could recover roughly 13,913 in monthly revenue.

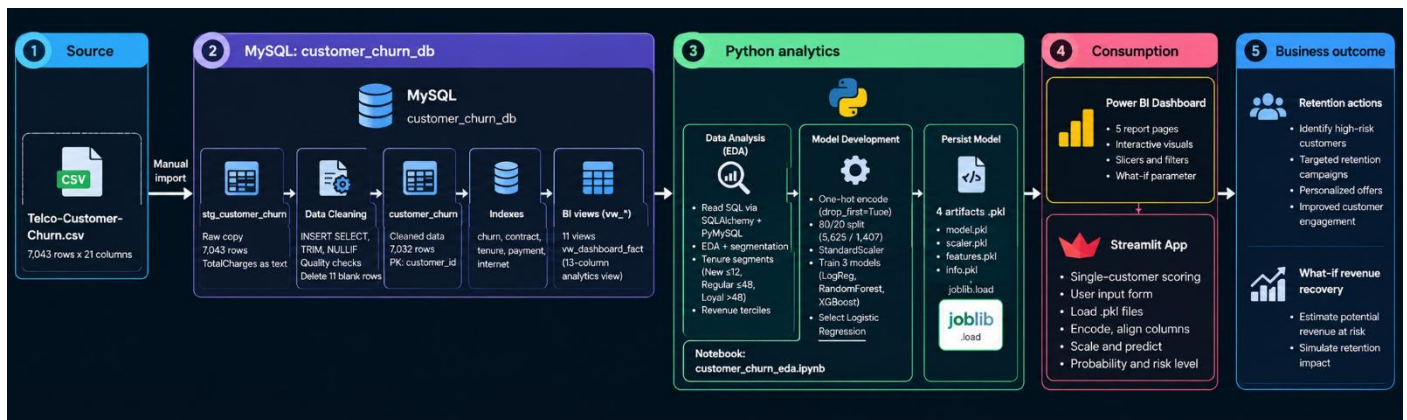


Fig1.1: End to End Data Flow

Data volume reconciliation

Checkpoint	Rows	Churn = Yes	Churn = No	Churn rate
Raw CSV / staging	7,043	1,869	5,174	26.54%
Cleaned table / views / notebook / Power BI	7,032	1,869	5,163	26.58%
Streamlit hard-coded text	7,043	1,869	5,174	"26.5%" ← inconsistent

HIGH LEVEL DESIGN (HLD)

The architecture utilizes a four-layer analytics stack governed by principles of a single source of truth, separation of concerns, and an interpretability-first approach for machine learning.

- **Source and Configuration:** Contains the raw CSV dataset and a Git-ignored config.env file holding database credentials.
- **Data Services Layer:** Relies on MySQL to house the raw staging table, the curated production table with five secondary indexes, and the 11 business views.
- **Analytics and ML Layer:** Powered by Python, pandas, and scikit-learn, this layer conducts segmentation and model comparison among Logistic Regression, Random Forest, and XGBoost. The Logistic Regression model is chosen for deployment over XGBoost due to its comparable ROC-AUC (0.8357 vs 0.8366), highest F1 score (0.581), and transparent feature coefficients.
- **Presentation Layer:** Exposes insights via a multi-page Power BI dashboard for executive reporting and a fast, Python-native Streamlit web app for single-customer churn prediction.

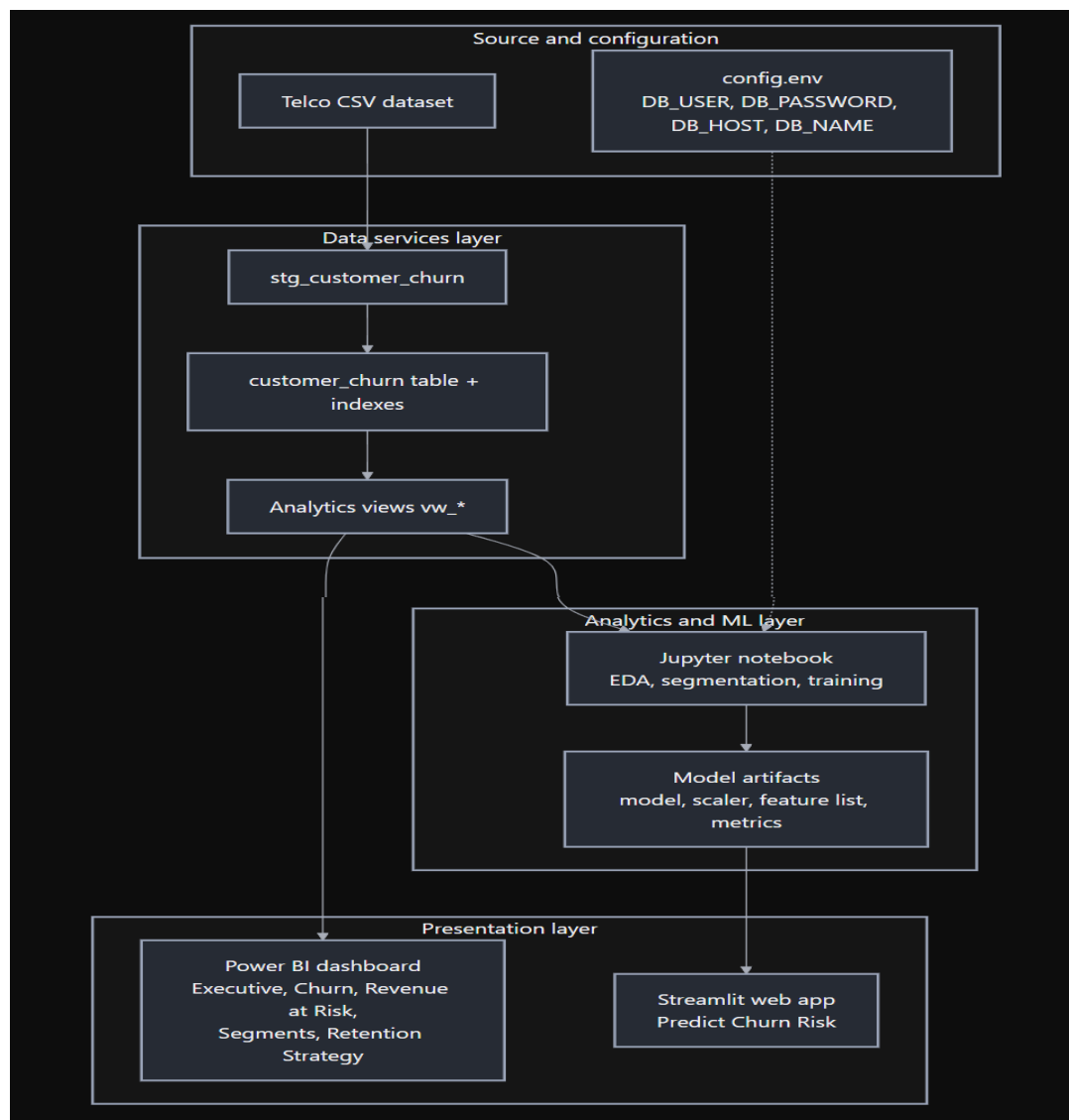


Fig 1.2: HLD

Component Catalogue

#	Component	Technology	Responsibility	Depends on
C1	Staging store	MySQL table <code>stg_customer_churn</code>	Land raw data untouched	CSV
C2	Curated store	MySQL table <code>customer_churn</code> + 5 indexes	Typed, de-duplicated, null-free customer records	C1
C3	Semantic layer	11 SQL views	Business definitions (segments, revenue lost, LTV)	C2
C4	Analytics notebook	Python, pandas, seaborn, scikit-learn, XGBoost	EDA, feature prep, model comparison, artifact export	C3, config.env
C5	Model artifacts	joblib <code>.pkl</code> x4	Portable trained model and preprocessing contract	C4
C6	BI dashboard	Power BI Desktop <code>.pbix</code>	Executive and diagnostic reporting, what-if	C3 (<code>vw_dashboard_fact</code>)
C7	Scoring app	Streamlit + Plotly	Interactive single-customer prediction	C5

Technology Stack

Layer	Choice	Rationale
Database	MySQL	Ubiquitous, adequate for 7K rows, supports views
Data access	SQLAlchemy 2 URL + PyMySQL	Safe credential handling, pandas <code>read_sql</code>
Analysis	pandas, NumPy, Matplotlib, Seaborn	Standard EDA toolchain
ML	scikit-learn (LogReg, RandomForest), XGBoost	Baseline interpretable model plus non-linear benchmarks
BI	Power BI	Business-user friendly dashboards, what-if parameter
App	Streamlit + Plotly	Fast Python-native UI with gauge visual
Persistence	joblib	Native for scikit-learn objects

Model Comparison

Model	Accuracy	Precision (churn)	Recall (churn)	F1	ROC-AUC
Logistic Regression	0.796	0.64	0.53	0.581	0.8357
XGBoost (300 trees, depth 4, lr 0.05)	0.792	0.64	0.51	0.563	0.8366
Random Forest (300 trees)	0.770	0.58	0.47	0.521	0.7996

HLD CONCLUSION

The system successfully delivers a clean, conventional analytics architecture that precisely isolates why and when customers churn. It serves as a highly effective analytical prototype, isolating month-to-month, fiber-optic, and short-tenure customers as the highest flight risks. However, it requires three architectural enhancements before production use: correcting functional data encoding bugs in the scoring application, establishing versioned automated pipelines to replace manual CSV loading, and instituting governance layers like authentication and monitoring.

LOW LEVEL DESIGN (LLD)

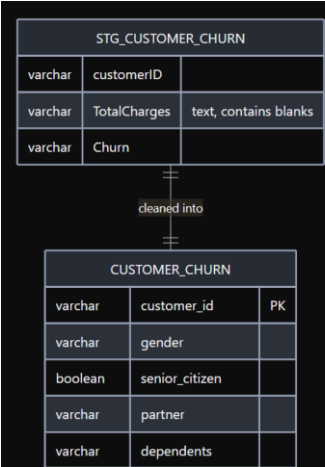
The low-level schema and execution strategy formalize the interactions between the database, the training environment, and the front-end application.

- **Database Execution:** Scripts run in a strict sequence: database creation, staging table generation, manual CSV import, cleaned table generation with QA domain checks, indexing, and view creation. The cleaning logic explicitly nullifies blank TotalCharges strings and deletes rows where charges are missing, dropping 11 records.
- **Python Notebook Design:** The EDA notebook extracts 7,032 rows from `vw_dashboard_fact` and prepares 14 features using one-hot encoding. It drops baseline categories (e.g., female, month-to-month contract, DSL) and scales the inputs using `StandardScaler`. The outputs are four files: `logistic_churn_model.pkl`, `scaler.pkl`, `model_features.pkl`, and `model_info.pkl`.
- **Streamlit Application Design:** The app dynamically loads the four artifacts, calculates total charges as `monthly_charges * tenure`, standardizes inputs, and predicts a probability score. A gauge visualization bands risks into Low (<0.40), Medium (≥ 0.40), High (≥ 0.60), and Very High (≥ 0.80) categories.

Repository Structure

```
Customer-Churn---Retention-Analytics-system/
├── .gitignore                # config.env, .env/
├── README.md
├── requirements.txt
├── dataset/Telco-Customer-Churn.csv
├── sql/
│   ├── create_db.sql        # CREATE DATABASE customer_churn_db
│   ├── create_table.sql     # stg_customer_churn (raw)
│   ├── create_cleaned_table.sql # customer_churn + clean + QA
│   ├── create_index.sql     # 5 indexes
│   └── create_BIviews.sql   # 11 views
├── notebooks/
│   ├── customer_churn_eda.ipynb # 94 cells: EDA + 3 models
│   ├── logistic_churn_model.pkl
│   ├── scaler.pkl
│   ├── model_features.pkl
│   └── model_info.pkl
├── powerbi/IBM Telco Customer Churn Analytics.pbix
├── streamlit/app.py         # loads ./notebooks/*.pkl (relative to CWD)
└── images/                  # 6 dashboard/app screenshots
```

Database Design



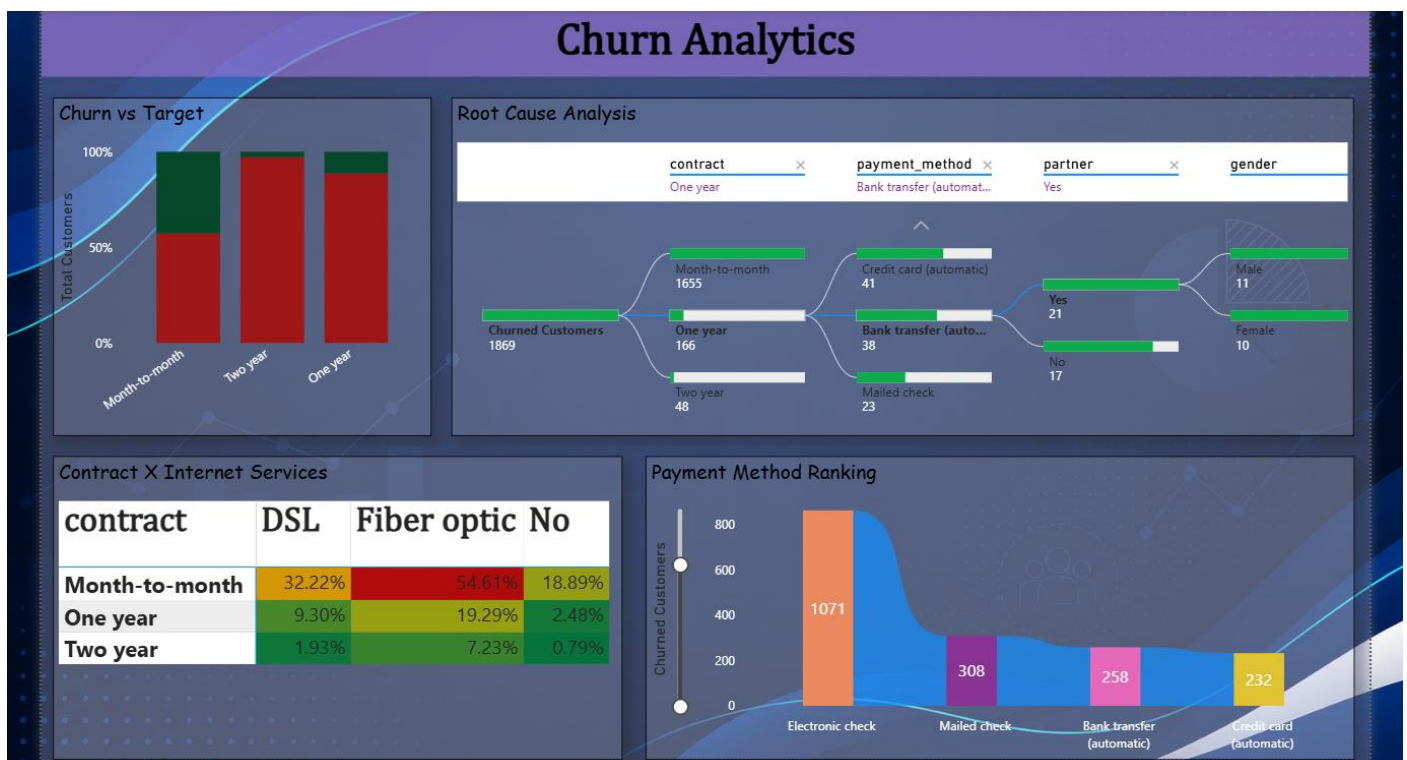
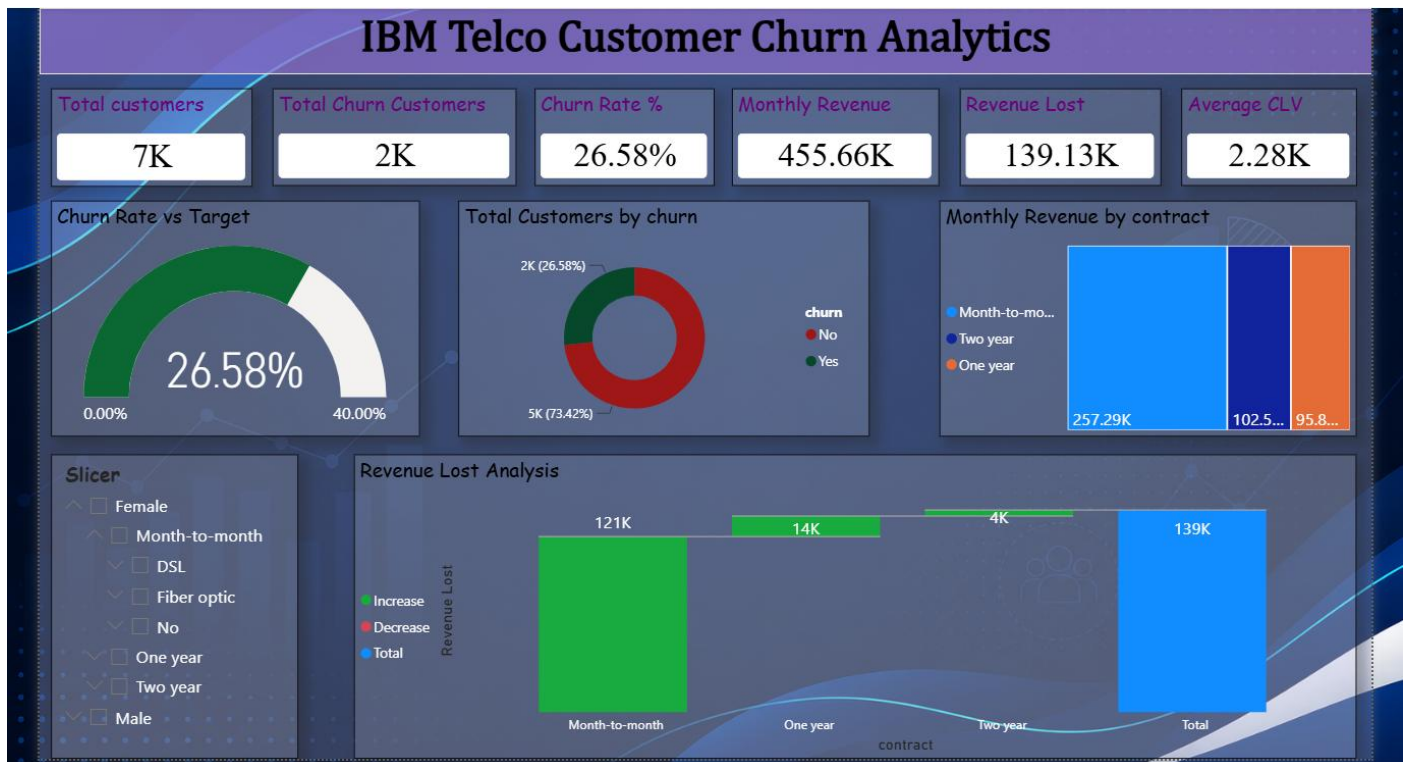
varchar	dependents	
int	tenure	
varchar	phone_service	
varchar	multiple_lines	
varchar	internet_service	
varchar	online_security	
varchar	online_backup	
varchar	device_protection	
varchar	tech_support	
varchar	streaming_tv	
varchar	streaming_movies	
varchar	contract	
varchar	paperless_billing	
varchar	payment_method	

varchar	payment_method	
decimal	monthly_charges	
decimal	total_charges	
varchar	churn	
timestamp	created_at	

View Catalogue

View	Grain	Logic (summary)	Consumer
<code>vw_customer_overview</code>	1 row	count, avg monthly charge, avg tenure	KPIs
<code>vw_churn_summary</code>	1 row	total, churned, churn rate %	KPIs
<code>vw_revenue_lost</code>	1 row	<code>SUM(monthly_charges) where churned</code>	KPI
<code>vw_contract_analysis</code>	contract	customers, churned	Churn Analysis
<code>vw_internet_analysis</code>	internet service	customers, churned	Churn Analysis
<code>vw_payment_analysis</code>	payment method	customers, churned	Churn Analysis
<code>vw_customer_segments</code>	customer	New ≤12, Regular ≤48, Loyal >48 months	Segments
<code>vw_revenue_segments</code>	customer	Low <35, Medium <70, High ≥70	Revenue at Risk
<code>vw_dashboard_fact</code>	customer	12 attributes + <code>churn_flag</code> (13 cols)	Power BI + Python
<code>vw_high_risk_revenue</code>	contract	monthly revenue & customers among churned	Revenue at Risk
<code>vw_customer_ltv</code>	customer	<code>monthly_charges × tenure</code>	CLV KPI

Power BI Design & Output

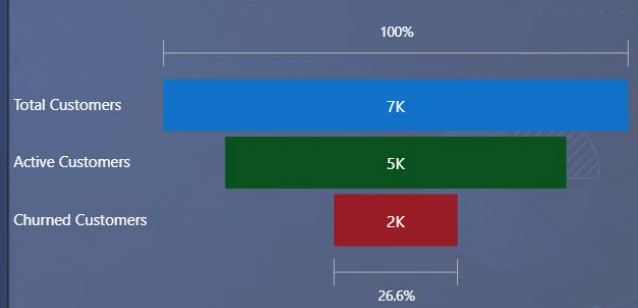


Revenue Risk Analysis

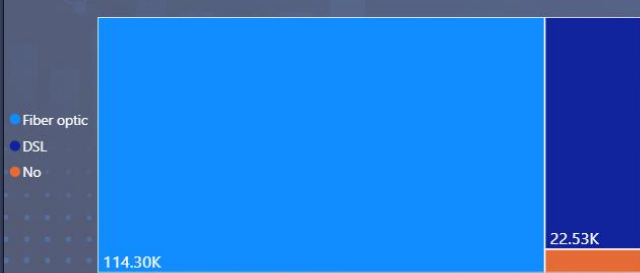
Revenue Lost Analysis



Customer Retention Funnel



Revenue Lost by Internet Service

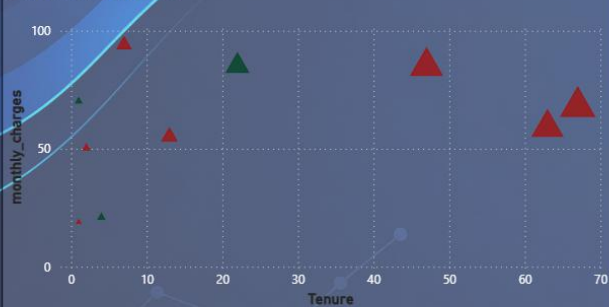


Segment Summary

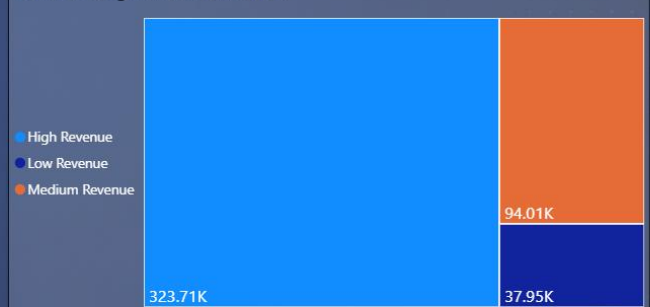
contract	Total Customers	Churn Customer %	Revenue Lost	Churn Rate %	Average CLV
Month-to-month	3875	0.43	120,847.10	42.71%	1,370.12
One year	1472	0.11	14,118.45	11.28%	3,031.89
Two year	1685	0.03	4,165.30	2.85%	3,728.76

Customer Segmentation

Customer Behaviour



Revenue Segment Distribution

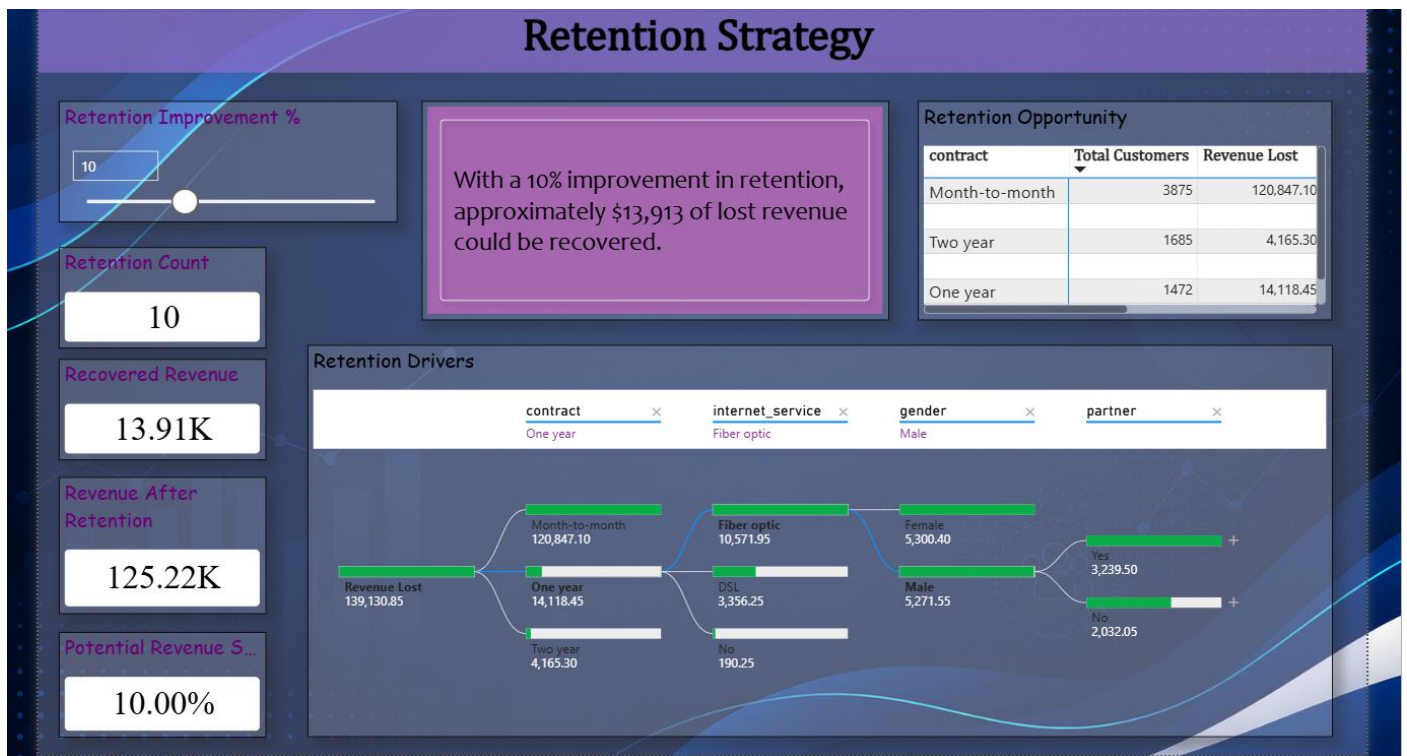


Segment Summary

Customer Segment	Total Customers	Churn Rate %	Monthly Revenue	Average CLV
Loyal	2239	9.51%	165,563.70	4,683.92
New	2175	47.68%	122,174.15	276.36
Regular	2618	23.64%	167,923.15	1,897.14

Customer Segment X Churn





LLD CONCLUSION

The repository's low-level layout provides an auditable path from table creation to artifact generation, utilizing a leak-free train/test split and sensible modeling choices. The primary LLD vulnerability lies in a critical defect (F1) within the Streamlit app's preprocessing. Because `pd.get_dummies(drop_first=True)` is applied to a single-row dataframe at runtime, six categorical inputs are silently dropped, causing drastically inaccurate user scores. Immediate remediation requires dropping the `drop_first` parameter in `app.py` or, ideally, migrating all preprocessing logic into a single scikit-learn Pipeline object.

Standard Operating Procedures (SOP)

The operational handbook defines repeatable steps for data engineers, data scientists, and application owners to manage the lifecycle of the churn system.

Role	Responsibility
Data Engineer / DBA	SOP-02, SOP-03, backups
Data Scientist	SOP-04, SOP-05
BI Developer	SOP-07
Application Owner	SOP-06, access control
Business Owner (Retention lead)	Approves model releases and thresholds; consumes dashboard

Prerequisites

- Python 3.10+ (a virtual environment is required)
- MySQL 8.x server and a client (MySQL Workbench or CLI)
- Power BI Desktop (Windows)
- Git
- A config.env file in the **repository root** with:

DB_USER=<read/write user for setup, read-only for analytics>

DB_PASSWORD=<password>

DB_HOST=localhost

DB_NAME=customer_churn_db

Never commit config.env; it is git-ignored.

SOP-01 - Environment setup

1. Clone or unzip the project and open a terminal in the repository root.
2. Create and activate a virtual environment: `python -m venv .venv` then `.venv\Scripts\activate` (Windows) or `source .venv/bin/activate`.
3. Install dependencies: `pip install -r requirements.txt` then `pip install sqlalchemy` (currently missing from the file; see F5).
4. Create config.env as shown above.
5. **Check:** `python -c "import pandas, sklearn, streamlit, sqlalchemy, pymysql, xgboost; print('ok')"` prints ok.

SOP-02 — Database setup and initial load

1. Run `sql/create_db.sql` (creates and selects `customer_churn_db`).
2. Run `sql/create_table.sql` (creates `stg_customer_churn`).
3. Import dataset/Telco-Customer-Churn.csv into `stg_customer_churn` (Workbench: right-click table → *Table Data Import Wizard* → choose the CSV → map columns by name).
4. **Check:** `SELECT COUNT(*) FROM stg_customer_churn;` returns **7,043**.
5. Run `sql/create_cleaned_table.sql`.
6. **Checks (all mandatory):**
 - `SELECT COUNT(*) FROM customer_churn;` → **7,032**
 - null count for `total_charges` → **0**
 - duplicate `customer_id` query → **no rows**
 - `SELECT DISTINCT churn` → **Yes, No** only
7. Run `sql/create_index.sql`, then `sql/create_BIviews.sql`.
8. **Check:** `SELECT * FROM vw_churn_summary;` → 7,032 / 1,869 / 26.58.

SOP-03 — Refreshing data

Use when a new extract arrives.

1. Back up: `mysqldump customer_churn_db > backup_<YYYYMMDD>.sql`.
2. `TRUNCATE stg_customer_churn;` then re-import the new CSV.
3. Record the staging row count.
4. Re-create `customer_churn` (drop and run `create_cleaned_table.sql`).
5. Compare counts to the previous load; investigate a change greater than $\pm 10\%$ before continuing.
6. Views update automatically. Re-run the validation checks in SOP-02 step 6 and 8.
7. Trigger SOP-07 (Power BI refresh) and assess whether SOP-04 (retraining) is due.

SOP-04 — Analysis and model retraining

Frequency: quarterly, or when monitoring (SOP-08) breaches a threshold.

1. Open `notebooks/customer_churn_eda.ipynb` from the notebooks folder (the config path is `../config.env`).
2. Run cells top to bottom (**Restart & Run All**).
3. Confirm the extract shape matches SOP-02 ((7032, 13) for the current dataset) and that there are no nulls.
4. Record metrics for all three models on the held-out test set (Accuracy, Precision, Recall, F1, ROC-AUC).

5. **Acceptance criteria for a new champion:** $\text{ROC-AUC} \geq \text{current} - 0.005$ **and** churn-class F1 $\geq$ current, with coefficients/feature importance that are business-plausible (longer tenure and longer contracts reduce risk).
6. If accepted, run the persistence cells (LogReg, scaler, features, info) **together** so all four files come from the same run.
7. Update model_info with the **measured** metrics; do not type them by hand from memory.

SOP-05 — Model release to the app

1. Copy the four .pkl files to a dated folder, e.g. releases/2026-09-23/, keeping the previous set.
2. Compute and record SHA-256 checksums of the files.
3. Overwrite notebooks/*.pkl used by the app.
4. Run app tests T5–T9 (LLD §3.8).
5. Business Owner signs off on the threshold and risk bands.
6. Tag the release in Git (model-vYYYY.MM.DD).
7. **Rollback:** restore the previous dated folder and restart the app (SOP-09 item 6).

SOP-06 — Running the Streamlit app

1. Activate the virtual environment.
2. **From the repository root** (the app loads ./notebooks/*.pkl relative to the working directory):
3. streamlit run streamlit/app.py
4. Open the URL printed in the terminal (default http://localhost:8501).
5. Smoke test before use (mandatory after any change):
 - Month-to-month, fiber optic, electronic check, tenure 2, monthly 95 → **high** risk
 - Two year, DSL, credit card, tenure 60, monthly 45 → **low** risk
 - If both show the same probability, the encoding defect (F1) is present. Stop and apply the LLD §3.7 fix.
6. Stop the app with Ctrl+C.

SOP-07 — Power BI refresh and publishing

1. Open powerbi/IBM Telco Customer Churn Analytics.pbix.
2. Confirm the MySQL data source (server, database customer_churn_db) and connect to **vw_dashboard_fact** and the supporting views.
3. Click **Refresh**; wait for completion without errors.
4. Reconcile the Executive Overview against SQL: total customers 7,032; churn rate 26.58%; revenue lost 139,130.85 (test T3, T4).
5. Check the what-if slider: 10% must show recovered revenue $\approx 13,913$.

6. Save. To share, publish to the Power BI Service and configure a scheduled refresh (a data gateway is needed for an on-premises MySQL).
7. Update screenshots in images/ if the layout changes.

Conclusion

The Customer Churn & Retention Analytics System successfully demonstrates how data analytics and machine learning can be used to solve a critical business problem. By combining SQL, Python, Machine Learning, Power BI, and Streamlit, the solution provides a complete analytical ecosystem for identifying churn risks, understanding customer behavior, and implementing retention strategies.

The system enables organizations to reduce customer attrition, protect recurring revenue, improve customer lifetime value, and make informed business decisions through actionable insights and predictive analytics. The project showcases strong capabilities in data engineering, business intelligence, predictive modeling, and application deployment, making it a valuable enterprise-level analytics solution.